LAPORAN

PENERAPAN TRANSAKSI DAN KONSISTENSI DATA PADA MYSQL

Disusun Untuk Memenuhi Tugas-3 P3

Mata Kuliah : Prak. Rekayasa & Arsitektur Basis Data

Dosen Pengampu:

Cecep Kurnia Sastradipraja, S.Kom., M.Kom.



Disusun Oleh:

Zulfa Annisa 2507049

Kelas B

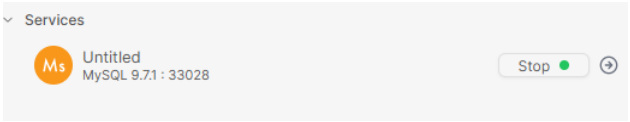
JURUSAN ILMU KOMPUTER

PROGRAM STUDI SISTEM INFORMASI

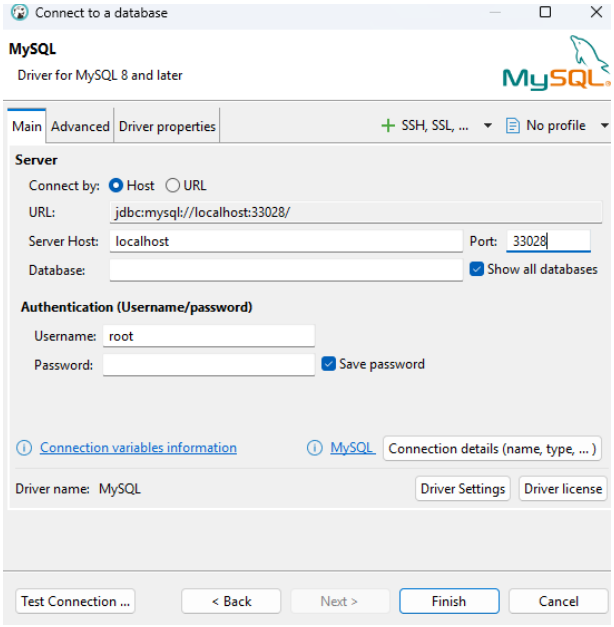
INSTITUT TEKNOLOGI GARUT

2026

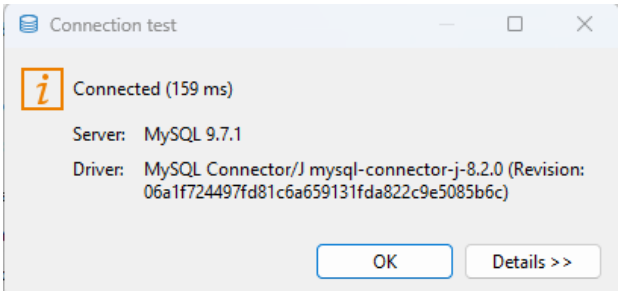
1. Mengaktifkan *service* MySQL 9.7.1 di DBngin pada port 33028 hingga berstatus *Running*.



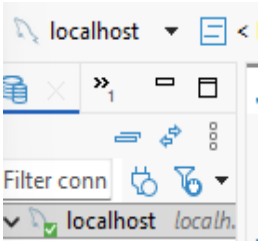
Membuat koneksi database MySQL baru di DBeaver dengan mengarahkan Server Host ke localhost dan Port 33028, lalu menguji koneksi (*Test Connection*) sampai muncul keterangan *Connected*.



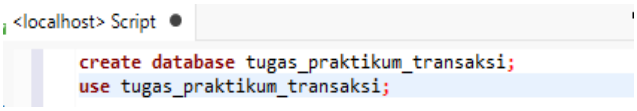
Jika sudah terkoneksi klik OK,



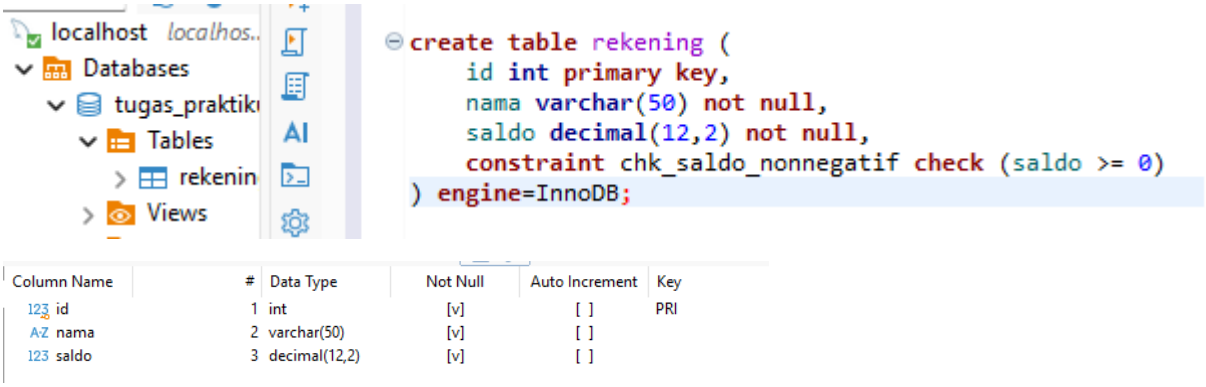
Selanjutnya, sebelum membuat SQL nya kita harus klik kanan localhost, lalu pilih SQL editor dan pilih New SQL script.



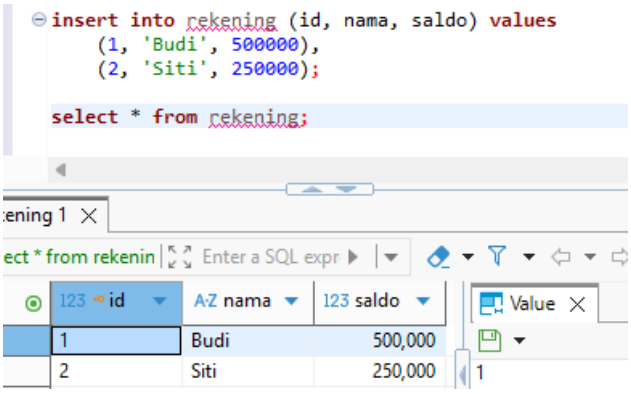
Jika sudah seperti ini, pertama kita buat databasenya terlebih dahulu lalu menjalankan perintah **create database tugas_praktikum_transaksi;** dan **use tugas_praktikum_transaksi;** untuk mengaktifkan database.



Selanjutnya, kita membuat struktur tabel rekening **ber-engine InnoDB dengan tipe data id (Primary Key), nama, saldo, serta batasan chk_saldo_nonnegatif.**



Memasukkan data awal saldo Budi (500.000) dan Siti (250.000) menggunakan **insert into**, lalu memverifikasi data awal **dengan select * from rekening;**.



2. Bagian I : Jawaban dari Latihan Mandiri

A. Transfer Berhasil

Memindahkan saldo Rp75.000 dari Budi ke Siti di dalam **start transaction;** dan meng-commit perubahan secara permanen lewat **commit;** , saldo akhir Budi adalah 425.000, Siti adalah 325.000.

```
-- Latihan A: Transfer Berhasil (Rp75.000)
select * from rekening;

start transaction;

update rekening
set saldo = saldo - 75000.00
where ID = 1;

update rekening
set saldo = saldo + 75000.00
where ID = 2;

select * from rekening;

commit;

select * from rekening;
```

select * from rekening | Enter a SQL expression to filter resul

Grid

	123 id	A-Z nama	123 saldo
1	1	Budi	425,000
2	2	Siti	325,000

B. Batalkan Perubahan

Memulai transaksi baru dan mengurangi saldo Budi sebesar Rp20.000, lalu memeriksa saldo sementara Budi yang terpotong menjadi 405.000.

```
-- Latihan B: Membatalkan Transaksi dengan ROLLBACK (Rp20.000)
select * from rekening;

start transaction;

update rekening
set saldo = saldo - 20000.00
where ID = 1;

select * from rekening;
```

select * from rekening | Enter a SQL exp

Grid

Text

	123 id	A-Z nama	123 saldo
1	1	Budi	405,000
2	2	Siti	325,000

Menjalankan perintah **rollback;** untuk membatalkan transaksi aktif, sehingga saldo Budi kembali utuh ke posisi sebelum transaksi (Rp425.000).

```
rollback;

select * from rekening;
```

select * from rekening | Enter a SQL expression to filter resul

	id	nama	saldo
1	1	Budi	425,000
2	2	Siti	325,000

C. SAVEPOINT

Memindahkan saldo Rp30.000 dari Budi ke Siti, kemudian menandai kondisi setelah transfer menggunakan **savepoint setelah_transfer;**

```
-- Latihan C: SAVEPOINT untuk pembatalan sebagian (Transfer Rp30.000, Tambahan Rp5.000)
start transaction;

update rekening
set saldo = saldo - 30000.00
where ID = 1;

update rekening
set saldo = saldo + 30000.00
where ID = 2;

SAVEPOINT setelah_transfer;

select * from rekening;
```

	id	nama	saldo
1	1	Budi	395,000
2	2	Siti	355,000

Melakukan simulasi pengurangan tambahan pada saldo Budi sebesar Rp5.000 setelah savepoint dibuat.

```
update rekening
set saldo = saldo - 5000.00
where ID = 1;
```

select * from rekening | Enter a SQL expression to filter result

	id	nama	saldo
1	1	Budi	390,000
2	2	Siti	355,000

Menjalankan **rollback to savepoint setelah_transfer;** untuk membatalkan pengurangan Rp5.000, lalu menyimpan transfer utama Rp30.000 secara permanen dengan **commit;.**

```
rollback to savepoint setelah_transfer;

commit;

select * from rekening;
```

id	nama	saldo
1	Budi	395,000
2	Siti	355,000

D. RELASE SAVEPOINT

Mengurangi saldo Budi sebesar Rp10.000 di dalam transaksi dan membuat titik simpan.

```
-- Latihan D: Lepaskan savepoint dengan RELEASE SAVEPOINT
start transaction;

update rekening
set saldo = saldo - 10000.00
where ID = 1;

savepoint sp_pemeriksaan;

select * from rekening;
```

	id	nama	saldo
1	1	Budi	385,000
2	2	Siti	355,000

Menambah saldo Siti Rp10.000, menghapus penanda savepoint menggunakan **release savepoint sp_pemeriksaan;**, lalu menyimpan seluruh transaksi ke database dengan **commit;**.

```
update rekening
set saldo = saldo + 10000.00
where ID = 2;

release savepoint sp_pemeriksaan;

commit;

select * from rekening;
```

ning 1

t * from rekening | Enter a SQL expression to filter results

id	nama	saldo
1	Budi	385,000
2	Siti	365,000

E. Analisis Risiko

,Menurut saya, Tanpa transaksi eksplisit, jika sistem terhenti setelah saldo pengirim berkurang, penambahan saldo penerima tidak dijalankan (pembaruan parsial). Hal ini menyebabkan data tidak seimbang dan uang hilang tanpa tercatat pada penerima.

3. Bagian II : Jawaban Evaluasi Pemahaman

1. **Apa yang dimaksud dengan transaksi pada basis data?**
2. **Apa perbedaan COMMIT dan ROLLBACK?**
3. **Mengapa transfer saldo sebaiknya menggunakan transaksi?**
4. **Apa fungsi SAVEPOINT?**
5. **Apa yang dilakukan ROLLBACK TO SAVEPOINT?**
6. **Apakah RELEASE SAVEPOINT sama dengan COMMIT? Jelaskan.**
7. **Mengapa tabel InnoDB digunakan dalam praktikum ini?**
8. **Apa yang perlu dilakukan jika hasil saldo tidak sesuai dengan hasil yang diharapkan?**

Jawab :

1. Sekumpulan perintah SQL yang diperlakukan sebagai satu unit kerja tunggal yang harus berhasil seluruhnya atau dibatalkan seluruhnya.
2. COMMIT menyimpan seluruh perubahan transaksi secara permanen, sedangkan ROLLBACK membatalkan seluruh perubahan transaksi yang belum di-commit.
3. Untuk menjaga konsistensi data (prinsip Atomicity) agar perubahan saldo pengirim dan penerima selalu diproses bersamaan tanpa risiko pembaruan parsial.
4. Membuat titik pemulihan di dalam transaksi aktif agar sebagian perubahan dapat dibatalkan jika diperlukan.

5. Membatalkan perubahan data setelah titik savepoint tertentu tanpa membatalkan transaksi utama sebelum savepoint tersebut.
6. Tidak sama. RELEASE SAVEPOINT hanya menghapus penanda savepoint, sedangkan COMMIT yang menyimpan perubahan transaksi secara permanen.
7. Karena InnoDB mendukung fitur transaksi (ACID), sedangkan engine non-transaksional tidak mendukung fitur tersebut.
8. Periksa urutan query, pastikan tidak ada COMMIT sebelum ROLLBACK, pastikan perintah berjalan dalam satu sesi koneksi yang sama, atau buat transaksi koreksi baru.